

Comparative Performance Analysis of Sorting Algorithms on Different Data Structures

Extended Academic Research Paper

Abstract

Sorting algorithms are fundamental to computer science and software engineering. This research investigates Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort, and Heap Sort on arrays and linked lists. The study evaluates theoretical and practical performance based on time complexity, memory consumption, stability, and scalability. The findings demonstrate that algorithm efficiency is strongly influenced by the underlying data structure and dataset characteristics.

1. Introduction

Sorting is one of the most frequently used operations in computing. Databases, operating systems, search engines, data analytics platforms, and scientific applications depend on efficient sorting mechanisms. The selection of an appropriate sorting algorithm can significantly improve execution speed and resource utilization.

2. Research Objectives

1. Compare major sorting algorithms. 2. Analyze behavior on arrays and linked lists. 3. Evaluate computational complexity. 4. Identify suitable algorithms for different dataset sizes. 5. Recommend best practices for practical applications.

3. Literature Review

Classical studies by Knuth, Cormen, and Sedgewick established the theoretical foundations of sorting algorithms. Modern research emphasizes practical performance, cache efficiency, memory locality, and hybrid approaches such as TimSort and IntroSort. Several studies conclude that Quick Sort performs exceptionally well in average cases, while Merge Sort is preferred when stability is required.

4. Research Methodology

The study compares six sorting algorithms across multiple data structures. Evaluation criteria include execution time, comparisons, swaps, memory usage, stability, adaptability to partially sorted data, and scalability.

5. Data Structures

Arrays provide direct indexing and excellent cache locality. Linked lists provide dynamic memory allocation and efficient insertion and deletion operations but lack constant-time random access. These characteristics affect sorting efficiency.

6. Sorting Algorithms

Bubble Sort repeatedly swaps adjacent elements. Selection Sort repeatedly selects the minimum element. Insertion Sort inserts elements into a growing sorted sequence. Merge Sort uses divide-and-conquer with merging. Quick Sort partitions around a pivot. Heap Sort utilizes a binary heap structure.

7. Complexity Analysis

Algorithm	Best Case	Average Case	Worst Case	Space
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$

8. Experimental Analysis

For small datasets (100 records), Insertion Sort showed competitive performance. For medium datasets (1,000 records), Quick Sort and Merge Sort significantly outperformed quadratic algorithms. For large datasets (10,000+ records), Quick Sort demonstrated excellent average-case efficiency, while Merge Sort maintained stable and predictable performance.

9. Performance on Arrays

Arrays favor algorithms that benefit from random access. Quick Sort generally achieved the best execution times due to efficient partitioning. Heap Sort provided consistent performance, while Merge Sort required additional memory.

10. Performance on Linked Lists

Merge Sort proved highly effective because merging linked lists requires only pointer manipulation. Quick Sort experienced additional overhead due to traversal requirements. Bubble Sort and Selection Sort remained inefficient.

11. Discussion

The analysis confirms that algorithm selection should depend on dataset size, memory constraints, stability requirements, and data structure characteristics. No single algorithm is universally optimal. Practical software systems often employ hybrid strategies to maximize performance.

12. Applications

Sorting algorithms are used in database indexing, operating systems, search engines, machine learning preprocessing, e-commerce systems, financial analytics, network routing, and scientific computing.

13. Future Scope

Future work can investigate parallel sorting algorithms, GPU-accelerated sorting, distributed computing environments, adaptive sorting techniques, and machine-learning based algorithm selection frameworks.

14. Conclusion

Quick Sort provides excellent average-case performance for arrays, Merge Sort is highly suitable for linked lists and stable sorting requirements, and Heap Sort offers strong worst-case guarantees. The choice of sorting algorithm should be guided by application-specific requirements rather than

theoretical complexity alone.

References

[1] Cormen, Leiserson, Rivest, and Stein, Introduction to Algorithms. [2] Donald E. Knuth, The Art of Computer Programming, Vol. 3. [3] Robert Sedgewick, Algorithms. [4] Mark Allen Weiss, Data Structures and Algorithm Analysis. [5] Goodrich and Tamassia, Data Structures and Algorithms.